부록 · Spring Boot — Java · Kotlin

Spring Framework와 Spring Boot 특징 비교

Java 웹 개발 및 JSP 수업 관점

구분	Spring Framework	Spring Boot
기본 개념	Java 애플리케이션 개발 프레임워크	Spring을 빠르고 편리하게 사용하도록 구성
설정	직접 설정할 부분이 비교적 많음	자동 설정(Auto Configuration) 제공
프로젝트 구성	상대적으로 복잡할 수 있음	간결하게 시작 가능
XML 설정	전통적으로 많이 사용	Java Config와 properties/yml 중심
의존성 관리	필요 라이브러리를 직접 조합	Starter로 간편하게 구성
웹 서버	외부 Tomcat 등에 WAR 배포 가능	내장 Tomcat으로 바로 실행 가능
대표 배포	WAR 방식이 전통적	실행 가능한 JAR 방식이 편리
설정 파일	XML, Java Config 등	application.properties 또는 .yml
DB 연결	DataSource 등을 직접 구성 가능	설정 파일과 자동 설정으로 간편
개발 속도	초기 설정에 시간이 더 필요할 수 있음	빠른 프로젝트 시작에 유리
REST API	지원	간편하게 구성 가능
JSP	지원	지원하나 JSP 관련 설정 필요
학습 관점	Spring 내부 구조 이해에 유리	초보자가 시작하기 쉬움
대표 장점	세밀한 설정과 높은 제어력	설정 간소화·빠른 개발·쉬운 실행
대표 단점	설정과 구성이 복잡해질 수 있음	자동 설정의 내부 동작 학습 필요
신규 프로젝트	직접 구성 가능	Spring Boot 기반으로 시작하기 편리

핵심 관계

Spring Boot는 Spring Framework를 대체하는 별개의 기술이라기보다, Spring Framework를 더 빠르고 편리하게 설정·실행하도록 도와주는 방식입니다.

수업용 정리

Servlet/JSP → Spring Framework → Spring Boot 순서로 발전 과정을 설명하고, 실습은 Spring Boot + Gradle + JSP + Bootstrap + MariaDB 구성으로 진행하면 전체 흐름을 이해하기 쉽습니다.

부록 · Spring Boot — Java · Kotlin

Java SpringBoot Kotlin SpringBoot 특징 비교표

Java + Spring Boot와 Kotlin + Spring Boot 특징 비교

Spring Boot는 동일하며 주 개발 언어의 차이를 중심으로 비교

구분	Java + Spring Boot	Kotlin + Spring Boot
개발 언어	Java	Kotlin
Spring Boot 기능	동일	동일
JVM 사용	O	O
코드 길이	상대적으로 김	짧고 간결함
문법	전통적이고 명시적	현대적이고 간결함
Getter/Setter	직접 작성 또는 Lombok 활용	프로퍼티로 간단히 처리
DTO 작성	코드량이 많아질 수 있음	data class로 매우 간단
Null 처리	개발자가 주의해서 관리	Null Safety 지원
타입 추론	지원	적극적으로 지원
함수형 문법	Stream/Lambda 지원	람다·컬렉션 처리에 편리
확장 함수	언어 기본 기능으로는 없음	지원
코루틴	Java 언어 기본 기능 아님	Coroutine 사용 가능
Java 라이브러리	사용 가능	Java 라이브러리와 높은 상호운용성
Spring 생태계	매우 성숙	Spring에서 공식 지원
기존 프로젝트	매우 많음	점차 확대
자료·예제	매우 많음	Java보다 상대적으로 적음
초보자 학습	자료가 많아 접근하기 좋음	Kotlin 문법을 추가로 익혀야 함
유지보수	안정적·보편적	코드량 감소에 유리
Gradle	사용 가능	Gradle Kotlin DSL 사용 가능
추천 용도	교육·기존 시스템·범용 개발	신규 프로젝트·간결한 백엔드 개발

코드 비교

Java

```java
public class Member {
    private Long id;
    private String name;
    private String email;

    // Getter / Setter
}
```

Kotlin

```kotlin
data class Member(
    var id: Long? = null,
    var name: String = "",
    var email: String = ""
)
```

수업 및 개발 관점 정리

Spring Boot + JSP + Bootstrap + MariaDB 수업에서는 Java + Spring Boot가 풍부한 자료와 전통적인 Java 구조를 설명하기 편리합니다. 신규 백엔드 프로젝트에서 코드량을 줄이고 Kotlin의 Null Safety, data class, 확장 함수 등을 활용하려면 Kotlin + Spring Boot가 좋은 선택입니다.

부록 · Spring Boot — Java · Kotlin

Maven과 Gradle 특징 비교

Spring Boot 프로젝트 및 수업 관점

구분	Maven	Gradle
대표 설정 파일	pom.xml	build.gradle / build.gradle.kts
설정 형식	XML	Groovy 또는 Kotlin DSL
코드량	비교적 많음	비교적 짧음
가독성	구조가 명확하지만 길어짐	간결하고 읽기 쉬움
학습 난이도	초보자에게 구조가 단순함	Gradle 문법을 조금 익혀야 함
빌드 속도	보통	증분 빌드·캐시 활용에 강점
의존성 관리	표준화되고 안정적	유연하고 간결함
커스터마이징	상대적으로 정형화됨	매우 유연함
Spring Boot	널리 사용됨	널리 사용됨
Android	일반적으로 사용하지 않음	표준 빌드 시스템
의존성 설정 예	<dependency>…</dependency>	implementation '…'
실행 명령	mvn spring-boot:run	./gradlew bootRun
Windows 실행	mvn spring-boot:run	.\gradlew.bat bootRun
Wrapper	mvnw / mvnw.cmd	gradlew / gradlew.bat
대표 장점	표준화·안정성·풍부한 사례	간결성·속도·유연성
대표 단점	XML 설정이 길어질 수 있음	빌드 스크립트 문법 학습 필요
수업용	전통적인 구조 설명에 좋음	짧은 설정으로 실습하기 좋음

간단한 설정 비교

Maven은 XML 기반이라 설정이 명확하지만 코드가 길어질 수 있습니다.

Gradle은 같은 의존성 설정을 더 짧게 작성할 수 있고, Gradle Wrapper를 함께 배포하면 학생 PC에 Gradle을 별도로 설치하지 않고도 프로젝트를 실행할 수 있습니다.

Spring Boot + JSP 수업에서는 Gradle + gradlew 방식이 설정이 간결하여 실습용으로 편리합니다.

부록 · Spring Boot — Java · Kotlin

Spring Boot 설정·빌드 파일 완전 정리

pom.xml · build.gradle(.kts) · gradlew · application.properties · application.yml

이 문서에서 고친 곳

- Kotlin + Spring Boot 용이라는 build.gradle.kts 예제에 kotlin 플러그인이 없고 java 만 있었습니다. kotlin("jvm")·plugin.spring·plugin.jpa 로 고쳤습니다.

1. 한눈에 보는 역할

파일	역할	빌드도구	핵심
pom.xml	의존성·플러그인·빌드 설정	Maven	XML 기반
build.gradle	의존성·플러그인·빌드 설정	Gradle	Groovy DSL
build.gradle.kts	의존성·플러그인·빌드 설정	Gradle	Kotlin DSL
gradlew / gradlew.bat	Gradle Wrapper 실행	Gradle	프로젝트 지정 Gradle 실행
application.properties	Spring Boot 실행환경 설정	공통	key=value
application.yml	Spring Boot 실행환경 설정	공통	계층형 YAML

핵심: pom.xml과 build.gradle(.kts)는 '빌드 설정', gradlew는 'Gradle 실행기', application.properties/yml은 '실행된 Spring Boot의 환경 설정'임.

2. pom.xml — Maven

POM(Project Object Model)은 Maven 프로젝트의 중심 파일임. Java 버전, Spring Boot 의존성, JPA, JDBC 드라이버, 테스트 및 빌드 플러그인을 정의함.

pom.xml 예제

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

Maven Wrapper 실행

```text
Windows: .\mvnw.cmd spring-boot:run
Linux/macOS: ./mvnw spring-boot:run
```

3. Gradle과 build.gradle.kts

Gradle은 Maven과 같은 빌드 자동화 도구임. Kotlin + Spring Boot에서는 build.gradle.kts(Kotlin DSL)를 사용하면 언어 통일성과 IDE 지원 측면에서 편리함.

build.gradle.kts 예제

```kotlin
plugins {
    kotlin("jvm") version "<Kotlin 버전>"
    kotlin("plugin.spring") version "<Kotlin 버전>"
    kotlin("plugin.jpa") version "<Kotlin 버전>"   // JPA 를 쓸 때
    id("org.springframework.boot") version "<Spring Boot 버전>"
    id("io.spring.dependency-management") version "<플러그인 버전>"
}
repositories { mavenCentral() }
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")
}
```

4. gradlew — Gradle Wrapper

gradlew는 설정 파일이 아니라 Gradle Wrapper 실행 스크립트임. 개발 PC에 Gradle을 별도로 설치하지 않아도 프로젝트가 지정한 Gradle 버전을 받아 동일한 환경으로 빌드할 수 있게 함.

정상 Wrapper 구조

```text
project/
├─ build.gradle.kts
├─ settings.gradle.kts
├─ gradlew
├─ gradlew.bat
└─ gradle/
   └─ wrapper/
      ├─ gradle-wrapper.jar
      └─ gradle-wrapper.properties
```

실행 명령

```text
Windows: .\gradlew bootRun
Linux/macOS: ./gradlew bootRun
```

org.gradle.wrapper.GradleWrapperMain 오류가 나오면 gradle-wrapper.jar 등 Wrapper 구성 파일 누락 여부를 우선 확인함.

5. Maven vs Gradle 비교

항목	Maven	Gradle
핵심 파일	pom.xml	build.gradle / build.gradle.kts
문법	XML	Groovy 또는 Kotlin DSL
Wrapper	mvnw	gradlew
설정량	상대적으로 많음	상대적으로 간결
Java 프로젝트	매우 보편적	매우 보편적
Kotlin 프로젝트	가능	특히 잘 어울림
Spring Boot	강력 지원	강력 지원

6. application.properties

src/main/resources에 두며 Spring Boot의 서버 포트, DB 접속정보, JPA 옵션 등 런타임 환경을 설정함.

MariaDB properties 예제

```properties
spring.application.name=member
server.port=8080
spring.datasource.url=jdbc:mariadb://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=1234
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

7. application.yml

properties와 목적은 같지만 계층 구조와 들여쓰기를 사용함. 설정이 많아질수록 관련 항목을 묶어 읽기 편함.

MariaDB YAML 예제

```yaml
spring:
  application:
    name: member
  datasource:
    url: jdbc:mariadb://localhost:3306/testdb
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
server:
  port: 8080
```

8. properties vs yml 비교

항목	application.properties	application.yml
형식	key=value	계층형 YAML
들여쓰기	중요하지 않음	매우 중요
입문	매우 쉬움	쉬움
설정이 많을 때	길어질 수 있음	구조적으로 정리하기 좋음
중복 표현	상대적으로 많음	적음
Spring Boot 지원	지원	지원
추천	단순/학습용	중대형 설정

둘을 동시에 사용할 수도 있으나 학습 단계에서는 한 형식으로 통일하는 것이 설정 충돌과 혼동을 줄이는 데 좋음.

9. Oracle · PostgreSQL · MariaDB 설정 비교

DB	JDBC URL 예	Driver
MariaDB	jdbc:mariadb://localhost:3306/testdb	org.mariadb.jdbc.Driver
PostgreSQL	jdbc:postgresql://localhost:5432/testdb	org.postgresql.Driver
Oracle	jdbc:oracle:thin:@localhost:1521/FREEPDB1	oracle.jdbc.OracleDriver

공통 application.yml 형태

```yaml
spring:
  datasource:
    url: <DB별 JDBC URL>
    username: <사용자>
    password: <비밀번호>
    driver-class-name: <DB별 Driver>
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

10. Java / Kotlin 권장 구성

용도	권장 구성
Java Spring Boot 학습	Maven + pom.xml + application.yml
Kotlin Spring Boot 주력	Gradle + build.gradle.kts + gradlew + application.yml
DB 입문	MariaDB
데이터·AI 서비스	PostgreSQL도 적극 학습
기업·공공 환경	Oracle도 학습

권장 흐름: build.gradle.kts 또는 pom.xml → 의존성 준비 → gradlew/mvnw로 Spring Boot 실행 → application.yml(properties) 로드 → DB 연결 → Controller → Service → Repository → JPA → Database.

11. 최종 암기

pom.xml / build.gradle.kts = 프로그램을 어떻게 빌드할지 설정함.

gradlew = 프로젝트가 요구하는 Gradle을 실행함.

application.properties / application.yml = 실행된 Spring Boot가 어떤 포트·DB·JPA 설정으로 동작할지 지정함.

부록 · Spring Boot — Java · Kotlin

Java와 Kotlin DTO 쉽게 비교하기

DTO = 데이터를 담아서 전달하는 상자

DTO란?

DTO(Data Transfer Object)는 이름, 이메일, 나이 같은 데이터를 담아서 Controller, Service 등 프로그램의 계층 사이에 전달하는 객체입니다. 쉽게 말하면 데이터를 담아 옮기는 '택배상자'입니다.

Java DTO

```java
public class MemberDto {

    private String name;
    private String email;
    private int age;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

Kotlin DTO

```kotlin
data class MemberDto(
    var name: String,
    var email: String,
    var age: Int
)
```

Java와 Kotlin 비교

Java	Kotlin
private String name	var name: String
private String email	var email: String
private int age	var age: Int
Getter 작성	자동 제공
Setter 작성	var이면 자동 제공
코드가 비교적 길음	data class로 매우 간결

핵심 문장

Java DTO = 데이터 + Getter/Setter를 비교적 길게 작성함

Kotlin DTO = data class로 같은 역할을 훨씬 짧게 작성함

Spring Boot에서의 역할

예를 들어 {"name":"홍길동","email":"hong@test.com","age":30} 같은 JSON 데이터가 들어오면 Java든 Kotlin이든 MemberDto라는 객체에 담아서 Controller 등에서 사용합니다.

부록 · Spring Boot — Java · Kotlin

DTO + JPA + Entity + Repository 특징 비교

Spring Boot 백엔드 구조 쉽게 이해하기

구분	DTO	JPA	Entity	Repository
한마디로	데이터 전달 상자	DB 영속성 기술	DB 테이블을 표현하는 객체	DB 작업 담당 창구
주요 목적	데이터 전달	객체와 DB 관리	테이블 데이터를 객체로 표현	CRUD 수행
DB와 관계	직접 연결 안 함	DB 영속성 관리	테이블과 매핑	Entity를 통해 DB 작업
테이블 매핑	X	매핑 표준 제공	O	Entity 사용
저장	자체 기능 없음	저장 기반 제공	저장할 데이터 상태 보유	save()
조회	자체 기능 없음	조회 기반 제공	조회 결과 표현	findAll(), findById()
삭제	자체 기능 없음	삭제 기반 제공	삭제 대상이 될 수 있음	delete(), deleteById()
대표 요소	data class 등	Persistence API	@Entity, @Id	JpaRepository
쉬운 비유	택배상자	물류 시스템	DB 자료를 나타내는 객체	DB 업무 담당자

전체 흐름

사용자 → DTO → Controller → Service → Entity → Repository → JPA → MariaDB

Kotlin 예제

DTO

```kotlin
data class MemberDto(
    val name: String,
    val email: String
)
```

Entity

```kotlin
@Entity
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String = ""
)
```

Repository

```kotlin
interface MemberRepository :
    JpaRepository<Member, Long>
```

기본 CRUD

```kotlin
repository.save(member)
repository.findAll()
repository.findById(1L)
repository.deleteById(1L)
```

핵심 암기

DTO = 전달

Entity = DB 데이터를 객체로 표현

Repository = DB 작업

JPA = 객체와 DB 사이의 영속성 작업을 위한 표준 기술

부록 · Spring Boot — Java · Kotlin

Spring Boot Java와 Kotlin JPA 쉽게 비교하기

JPA = 객체와 데이터베이스 테이블을 연결해 주는 기술

1. JPA란?

JPA는 Java/Kotlin 객체와 DB 테이블을 연결하여 데이터를 저장·조회·수정·삭제하기 쉽게 해주는 기술입니다. 쉽게 말하면 '객체와 DB 사이의 연결다리'입니다.

2. Java + Spring Boot JPA Entity

```java
@Entity
@Table(name = "member")
public class Member {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // Getter / Setter
}
```

3. Kotlin + Spring Boot JPA Entity

```kotlin
@Entity
@Table(name = "member")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,

    var name: String = "",
    var email: String = ""
)
```

4. 주요 어노테이션

어노테이션	쉬운 의미
@Entity	이 클래스는 DB 테이블과 연결됨
@Table(name="member")	member 테이블과 연결
@Id	기본키(PK) 지정
@GeneratedValue	ID 값을 DB에서 자동 생성

5. Repository 비교

Java

```java
public interface MemberRepository extends JpaRepository<Member, Long> {
}
```

Kotlin

```kotlin
interface MemberRepository : JpaRepository<Member, Long>
```

6. 기본 DB 작업

JPA 메서드	기능
save()	저장 또는 수정
findAll()	전체 조회
findById()	한 건 조회
deleteById()	삭제

7. JDBC와 JPA 차이

```yaml
JDBC: Java/Kotlin → SQL 직접 작성 → MariaDB
JPA: Java/Kotlin 객체 → JPA가 SQL 처리 → MariaDB
```

핵심 정리

DTO = 데이터를 전달하는 상자

Entity = DB 테이블과 연결되는 객체

Repository = DB 저장·조회·수정·삭제를 담당

처음에는 Entity → Repository → Controller → MariaDB 순서로 익히고, 이후 Service와 DTO를 추가하면 이해하기 쉽습니다.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · Spring Boot — Java · Kotlin

JPA + Entity + Repository 간단 예제

Spring Boot Java와 Kotlin 비교

구분	Java + Spring Boot	Kotlin + Spring Boot
Entity 선언	@Entity public class Member	@Entity class Member
ID	private Long id	var id: Long? = null
이름	private String name	var name: String = ""
Getter/Setter	직접 작성 또는 도구 활용	var 사용 시 별도 작성 불필요
Repository	extends JpaRepository	: JpaRepository
저장	repository.save(member)	repository.save(member)
전체 조회	repository.findAll()	repository.findAll()
한 건 조회	repository.findById(1L)	repository.findById(1L)
삭제	repository.deleteById(1L)	repository.deleteById(1L)
특징	명시적이고 코드가 비교적 김	간결하고 코드량이 적음

Java Entity

```java
@Entity
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    // Getter / Setter
}
```

Kotlin Entity

```kotlin
@Entity
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = ""
)
```

Java Repository

```java
public interface MemberRepository
        extends JpaRepository<Member, Long> {
}
```

Kotlin Repository

```kotlin
interface MemberRepository :
    JpaRepository<Member, Long>
```

기본 CRUD

```text
save()       → 저장
findAll()    → 전체 조회
findById()   → 한 건 조회
deleteById() → 삭제
```

핵심 관계

JPA → Entity → Repository → MariaDB

JPA = 객체와 DB의 영속성을 관리하는 표준

Entity = DB 테이블을 객체로 표현

Repository = Entity를 저장·조회·삭제하는 DB 작업 창구

부록 · Spring Boot — Java · Kotlin

Spring Boot Java + Kotlin JPA · DTO · Repository · Entity 연동 예제

Oracle · PostgreSQL · MariaDB 비교 및 REST API 실습

이 문서에서 고친 곳

- Java 예제가 Lombok 을 쓰는데 의존성 안내가 없어, 그대로 따라 하면 컴파일 오류가 납니다. 아래 의존성을 먼저 넣습니다.

```xml
<!-- pom.xml 의 <dependencies> 안에 -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

```kotlin
// build.gradle.kts 를 쓰는 Java 프로젝트라면
compileOnly("org.projectlombok:lombok")
annotationProcessor("org.projectlombok:lombok")
```

1. 전체 구조

Spring Boot에서 데이터베이스가 바뀌더라도 Entity → Repository → Service → DTO → Controller라는 기본 구조는 거의 동일함.

Client → REST Controller → Service → DTO ↔ Entity → Repository → Spring Data JPA/Hibernate → Oracle · PostgreSQL · MariaDB

2. Oracle · PostgreSQL · MariaDB 비교

항목	Oracle	PostgreSQL	MariaDB
성격	기업용 상용 DB 중심	강력한 오픈소스 DB	MySQL 계열 오픈소스 DB
Spring Boot/JPA	매우 좋음	매우 좋음	매우 좋음
자동 증가	Identity / Sequence	Identity / Sequence	AUTO_INCREMENT
JSON	지원	매우 강력	지원
비용	상용 환경 라이선스 주의	무료	무료
기업/공공	강점	사용 증가	사용
웹 서비스	좋음	매우 좋음	매우 편함
AI/데이터	좋음	추천	좋음
초보 학습	보통	보통	쉬움

3. 권장 프로젝트 구조

```
src/main/
├── java 또는 kotlin/com/example/demo/
│   ├── controller/MemberController
│   ├── service/MemberService
│   ├── repository/MemberRepository
│   ├── entity/Member
│   ├── dto/MemberDto
│   └── DemoApplication
└── resources/application.yml
```

4. Entity

Java

```java
@Entity
@Table(name = "member")
@Getter @Setter @NoArgsConstructor
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

Kotlin

```kotlin
@Entity
@Table(name = "member")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String = "",
    var age: Int = 0
)
```

Entity는 데이터베이스 테이블과 매핑되는 객체임.

5. DTO

Java

```java
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class MemberDto {
    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

Kotlin

```kotlin
data class MemberDto(
    val id: Long? = null,
    val name: String,
    val email: String,
    val age: Int
)
```

DTO는 API 계층에서 데이터를 주고받기 위한 객체임. Entity를 API에 직접 노출하지 않는 구조를 만들 때 유용함.

6. Repository

Java

```java
public interface MemberRepository
        extends JpaRepository<Member, Long> {
    List<Member> findByName(String name);
}
```

Kotlin

```kotlin
interface MemberRepository : JpaRepository<Member, Long> {
    fun findByName(name: String): List<Member>
}
```

JpaRepository를 상속하면 save(), findById(), findAll(), delete(), deleteById(), count() 등의 기본 CRUD 기능을 바로 사용할 수 있음.

7. Service - DTO ↔ Entity 변환

Java

```java
@Service
@RequiredArgsConstructor
public class MemberService {
    private final MemberRepository repository;

    public MemberDto save(MemberDto dto) {
        Member member = new Member();
        member.setName(dto.getName());
        member.setEmail(dto.getEmail());
        member.setAge(dto.getAge());

        Member saved = repository.save(member);
        return new MemberDto(saved.getId(), saved.getName(),
                saved.getEmail(), saved.getAge());
    }

    public List<MemberDto> findAll() {
        return repository.findAll().stream()
            .map(m -> new MemberDto(m.getId(), m.getName(),
                    m.getEmail(), m.getAge()))
            .toList();
    }
}
```

Kotlin

```kotlin
@Service
class MemberService(
    private val repository: MemberRepository
) {
    fun save(dto: MemberDto): MemberDto {
        val saved = repository.save(
            Member(name = dto.name, email = dto.email, age = dto.age)
        )
        return MemberDto(saved.id, saved.name, saved.email, saved.age)
    }

    fun findAll(): List<MemberDto> =
        repository.findAll().map {
            MemberDto(it.id, it.name, it.email, it.age)
        }
}
```

8. REST Controller

Java

```java
@RestController
@RequestMapping("/api/members")
@RequiredArgsConstructor
public class MemberController {
    private final MemberService service;

    @PostMapping
    public MemberDto save(@RequestBody MemberDto dto) {
        return service.save(dto);
    }

    @GetMapping
    public List<MemberDto> findAll() {
        return service.findAll();
    }
}
```

Kotlin

```kotlin
@RestController
@RequestMapping("/api/members")
class MemberController(
    private val service: MemberService
) {
    @PostMapping
    fun save(@RequestBody dto: MemberDto) = service.save(dto)

    @GetMapping
    fun findAll() = service.findAll()
}
```

등록 요청 JSON

```text
POST /api/members

{
  "name": "홍길동",
  "email": "hong@test.com",
  "age": 35
}
```

9. 데이터베이스 연결 설정

MariaDB - application.yml

```yaml
spring:
  datasource:
    url: jdbc:mariadb://localhost:3306/testdb
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

PostgreSQL - application.yml

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/testdb
    username: postgres
    password: 1234
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

Oracle - application.yml

```yaml
spring:
  datasource:
    url: jdbc:oracle:thin:@localhost:1521/FREEPDB1
    username: testuser
    password: 1234
    driver-class-name: oracle.jdbc.OracleDriver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

Gradle JDBC 드라이버 예

```text
// MariaDB
runtimeOnly("org.mariadb.jdbc:mariadb-java-client")

// PostgreSQL
runtimeOnly("org.postgresql:postgresql")

// Oracle
runtimeOnly("com.oracle.database.jdbc:ojdbc11")
```

Oracle의 키 생성 전략은 실제 Oracle 버전과 테이블 설계에 따라 IDENTITY 또는 SEQUENCE를 선택함.

10. Java와 Kotlin 비교표

항목	Spring Boot Java	Spring Boot Kotlin
코드량	상대적으로 많음	적음
DTO	일반 클래스/record/Lombok	data class가 편리
Getter/Setter	직접 작성 또는 Lombok	프로퍼티로 간결
Null 처리	주의 필요	언어 차원에서 강함
Repository	간단	더 간결
Controller	간단	더 간결
JPA 생태계	매우 성숙	매우 좋음
학습자료	매우 많음	많음
기존 기업 프로젝트	강점	증가 추세
신규 개발	좋음	추천

11. 핵심 정리

REST API 정석 구조: Controller → Service → DTO ↔ Entity → Repository → JPA → Database

DB를 MariaDB에서 PostgreSQL 또는 Oracle로 변경해도 애플리케이션 계층 구조는 대부분 유지할 수 있음. JDBC 드라이버와 datasource 설정, 키 생성 방식, DB 고유 SQL/데이터 타입 등의 차이를 주로 확인하면 됨.

학습 순서는 MariaDB로 CRUD를 완성한 뒤 PostgreSQL과 Oracle로 DB만 교체해 동일 API를 실행해 보는 방식이 이해하기 쉬움.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · Spring Boot — Java · Kotlin

Spring Boot REST API 정석 입문

DTO + JPA + Entity + Repository : Java와 Kotlin 쉬운 비교

1. REST API 기본 흐름

JSON → Controller → DTO → Service → Entity → Repository → JPA → MariaDB

2. 핵심 개념 비교

구분	쉬운 뜻	대표 예
DTO	데이터를 주고받는 상자	MemberDto
Entity	DB 테이블을 객체로 표현	Member
Repository	DB 저장·조회·수정·삭제 담당	MemberRepository
JPA	객체와 DB의 영속성을 연결·관리하는 표준 기술	save(), findAll() 등의 기반

3. Java + Spring Boot

Java DTO

```java
public class MemberDto {
    private String name;
    private String email;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Java Entity

```java
@Entity
@Table(name = "member")
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Java Repository

```java
public interface MemberRepository
        extends JpaRepository<Member, Long> {
}
```

Java Service

```java
@Service
public class MemberService {
    private final MemberRepository repository;

    public MemberService(MemberRepository repository) {
        this.repository = repository;
    }

    public Member save(MemberDto dto) {
        Member member = new Member();
        member.setName(dto.getName());
        member.setEmail(dto.getEmail());
        return repository.save(member);
    }

    public List<Member> findAll() {
        return repository.findAll();
    }
}
```

Java Controller

```java
@RestController
@RequestMapping("/members")
public class MemberController {
    private final MemberService service;

    public MemberController(MemberService service) {
        this.service = service;
    }

    @PostMapping
    public Member save(@RequestBody MemberDto dto) {
        return service.save(dto);
    }

    @GetMapping
    public List<Member> findAll() {
        return service.findAll();
    }
}
```

4. Kotlin + Spring Boot

Kotlin DTO

```kotlin
data class MemberDto(
    val name: String,
    val email: String
)
```

Kotlin Entity

```kotlin
@Entity
@Table(name = "member")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String = ""
)
```

Kotlin Repository

```kotlin
interface MemberRepository :
    JpaRepository<Member, Long>
```

Kotlin Service

```kotlin
@Service
class MemberService(
    private val repository: MemberRepository
) {
    fun save(dto: MemberDto): Member {
        val member = Member(
            name = dto.name,
            email = dto.email
        )
        return repository.save(member)
    }

    fun findAll(): List<Member> =
        repository.findAll()
}
```

Kotlin Controller

```kotlin
@RestController
@RequestMapping("/members")
class MemberController(
    private val service: MemberService
) {
    @PostMapping
    fun save(@RequestBody dto: MemberDto): Member =
        service.save(dto)

    @GetMapping
    fun findAll(): List<Member> =
        service.findAll()
}
```

5. Java와 Kotlin 비교

항목	Java + Spring Boot	Kotlin + Spring Boot
DTO	class + getter/setter	data class
Entity	@Entity class	@Entity class
Repository	extends JpaRepository	: JpaRepository
Service	@Service	@Service
Controller	@RestController	@RestController
JPA 저장	repository.save()	repository.save()
JPA 조회	repository.findAll()	repository.findAll()
구조	동일	동일
코드량	비교적 많음	간결함

6. REST API 동작 예

POST /members 요청 JSON

```json
{
  "name": "홍길동",
  "email": "hong@test.com"
}
```

POST /members → Controller → DTO → Service → Entity → Repository → JPA → MariaDB

7. REST API 정석 구조

```text
dto/
 ├─ MemberRequestDto
 └─ MemberResponseDto

entity/
 └─ Member

repository/
 └─ MemberRepository

service/
 └─ MemberService

controller/
 └─ MemberController
```

학습 순서: POST 등록 → GET 전체조회 → GET 한 건 조회 → PUT 수정 → DELETE 삭제

8. 핵심 암기

DTO = API 데이터를 주고받는 상자

Entity = DB 테이블을 객체로 표현

Repository = Entity의 CRUD 담당

JPA = 객체와 DB의 영속성을 연결·관리하는 표준

정석 흐름 = DTO → Controller → Service → Entity → Repository → JPA → MariaDB

부록 · Spring Boot — Java · Kotlin

Spring Boot CRUD 완성 프로젝트

Java + Kotlin · JPA · DTO · Entity · Repository · Service · REST Controller · MariaDB

이 문서에서 고친 곳

- Java 예제가 Lombok(@Getter·@Setter 등)을 쓰는데 의존성 안내가 없어, 그대로 따라 하면 컴파일 오류가 납니다. 아래 의존성을 먼저 넣습니다.

```xml
<!-- pom.xml 의 <dependencies> 안에 -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

```kotlin
// build.gradle.kts 를 쓰는 Java 프로젝트라면
compileOnly("org.projectlombok:lombok")
annotationProcessor("org.projectlombok:lombok")
```

1. CRUD와 REST API

CRUD	SQL	HTTP	예제	기능
Create	INSERT	POST	/api/members	회원 등록
Read	SELECT	GET	/api/members, /api/members/1	전체/단건 조회
Update	UPDATE	PUT	/api/members/1	회원 수정
Delete	DELETE	DELETE	/api/members/1	회원 삭제

전체 흐름: Client → Controller → Service → DTO ↔ Entity → Repository → Spring Data JPA/Hibernate → MariaDB

2. 프로젝트 폴더 구조

Java

```text
member-java/
├─ pom.xml
├─ mvnw / mvnw.cmd
└─ src/main/
   ├─ java/com/example/member/
   │  ├─ MemberApplication.java
   │  ├─ controller/MemberController.java
   │  ├─ service/MemberService.java
   │  ├─ repository/MemberRepository.java
   │  ├─ entity/Member.java
   │  └─ dto/MemberDto.java
   └─ resources/application.yml
```

Kotlin

```text
member-kotlin/
├─ build.gradle.kts
├─ settings.gradle.kts
├─ gradlew / gradlew.bat
├─ gradle/wrapper/
└─ src/main/
   ├─ kotlin/com/example/member/
   │  ├─ MemberApplication.kt
   │  ├─ controller/MemberController.kt
   │  ├─ service/MemberService.kt
   │  ├─ repository/MemberRepository.kt
   │  ├─ entity/Member.kt
   │  └─ dto/MemberDto.kt
   └─ resources/application.yml
```

3. MariaDB 준비

DB 생성

```sql
CREATE DATABASE memberdb CHARACTER SET utf8mb4;
USE memberdb;
```

예제에서는 spring.jpa.hibernate.ddl-auto=update를 사용하여 Entity를 기준으로 테이블을 생성·갱신함. 운영환경에서는 스키마 마이그레이션 도구 사용을 검토하는 것이 좋음.

4. Entity

Java - Member.java

```java
@Entity
@Table(name = "members")
@Getter @Setter @NoArgsConstructor
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String email;
    private Integer age;
}
```

Kotlin - Member.kt

```kotlin
@Entity
@Table(name = "members")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,

    @Column(nullable = false)
    var name: String = "",

    var email: String = "",
    var age: Int = 0
)
```

Entity는 Java/Kotlin 객체와 DB 테이블을 연결함. @Id는 기본키, @GeneratedValue는 키 자동 생성 전략을 의미함.

5. DTO

Java - MemberDto.java

```java
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class MemberDto {
    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

Kotlin - MemberDto.kt

```kotlin
data class MemberDto(
    val id: Long? = null,
    val name: String,
    val email: String,
    val age: Int
)
```

DTO는 API 입출력 데이터를 전달하는 객체임. Entity를 외부 API에 직접 노출하지 않고 필요한 필드만 전달하는 구조를 만들 수 있음.

6. Repository

Java

```java
public interface MemberRepository
        extends JpaRepository<Member, Long> {
}
```

Kotlin

```kotlin
interface MemberRepository :
    JpaRepository<Member, Long>
```

JpaRepository가 save(), findAll(), findById(), existsById(), deleteById() 등 기본 CRUD를 제공함.

7. Service — CRUD 핵심

Java - MemberService.java

```java
@Service
@RequiredArgsConstructor
public class MemberService {
    private final MemberRepository repository;

    public MemberDto create(MemberDto dto) {
        Member m = new Member();
        m.setName(dto.getName());
        m.setEmail(dto.getEmail());
        m.setAge(dto.getAge());
        return toDto(repository.save(m));
    }

    public List<MemberDto> findAll() {
        return repository.findAll().stream()
                .map(this::toDto).toList();
    }

    public MemberDto findById(Long id) {
        Member m = repository.findById(id)
                .orElseThrow(() ->
                    new RuntimeException("회원이 없습니다."));
        return toDto(m);
    }

    public MemberDto update(Long id, MemberDto dto) {
        Member m = repository.findById(id)
                .orElseThrow(() ->
                    new RuntimeException("회원이 없습니다."));
        m.setName(dto.getName());
        m.setEmail(dto.getEmail());
        m.setAge(dto.getAge());
        return toDto(repository.save(m));
    }

    public void delete(Long id) {
        if (!repository.existsById(id))
            throw new RuntimeException("회원이 없습니다.");
        repository.deleteById(id);
    }

    private MemberDto toDto(Member m) {
        return new MemberDto(
            m.getId(), m.getName(), m.getEmail(), m.getAge());
    }
}
```

Kotlin - MemberService.kt

```kotlin
@Service
class MemberService(
    private val repository: MemberRepository
) {
    fun create(dto: MemberDto): MemberDto =
        repository.save(
            Member(name=dto.name, email=dto.email, age=dto.age)
        ).toDto()

    fun findAll(): List<MemberDto> =
        repository.findAll().map { it.toDto() }

    fun findById(id: Long): MemberDto =
        repository.findById(id)
            .orElseThrow { RuntimeException("회원이 없습니다.") }
            .toDto()

    fun update(id: Long, dto: MemberDto): MemberDto {
        val m = repository.findById(id)
            .orElseThrow { RuntimeException("회원이 없습니다.") }
        m.name = dto.name
        m.email = dto.email
        m.age = dto.age
        return repository.save(m).toDto()
    }

    fun delete(id: Long) {
        if (!repository.existsById(id))
            throw RuntimeException("회원이 없습니다.")
        repository.deleteById(id)
    }

    private fun Member.toDto() =
        MemberDto(id, name, email, age)
}
```

8. REST Controller

Java - MemberController.java

```java
@RestController
@RequestMapping("/api/members")
@RequiredArgsConstructor
public class MemberController {
    private final MemberService service;

    @PostMapping
    public MemberDto create(@RequestBody MemberDto dto) {
        return service.create(dto);
    }

    @GetMapping
    public List<MemberDto> findAll() {
        return service.findAll();
    }

    @GetMapping("/{id}")
    public MemberDto findById(@PathVariable Long id) {
        return service.findById(id);
    }

    @PutMapping("/{id}")
    public MemberDto update(
            @PathVariable Long id,
            @RequestBody MemberDto dto) {
        return service.update(id, dto);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

Kotlin - MemberController.kt

```kotlin
@RestController
@RequestMapping("/api/members")
class MemberController(
    private val service: MemberService
) {
    @PostMapping
    fun create(@RequestBody dto: MemberDto) =
        service.create(dto)

    @GetMapping
    fun findAll() = service.findAll()

    @GetMapping("/{id}")
    fun findById(@PathVariable id: Long) =
        service.findById(id)

    @PutMapping("/{id}")
    fun update(
        @PathVariable id: Long,
        @RequestBody dto: MemberDto
    ) = service.update(id, dto)

    @DeleteMapping("/{id}")
    fun delete(@PathVariable id: Long) =
        service.delete(id)
}
```

9. application.yml

MariaDB 연결

```yaml
spring:
  datasource:
    url: jdbc:mariadb://localhost:3306/memberdb
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true

server:
  port: 8080
```

10. 실제 CRUD 테스트

CREATE

```text
POST http://localhost:8080/api/members

{
  "name": "홍길동",
  "email": "hong@test.com",
  "age": 35
}
```

READ

```text
GET http://localhost:8080/api/members
GET http://localhost:8080/api/members/1
```

UPDATE

```text
PUT http://localhost:8080/api/members/1

{
  "name": "김길동",
  "email": "kim@test.com",
  "age": 40
}
```

DELETE

```sql
DELETE http://localhost:8080/api/members/1
```

11. Java vs Kotlin 상세 비교

항목	Java	Kotlin
Entity	명시적이며 코드가 다소 김	프로퍼티로 간결
DTO	클래스/record/Lombok	data class가 편리
Repository	매우 간단	매우 간단
Service	Stream·getter/setter 등 코드 증가	컬렉션 함수와 프로퍼티로 간결
Controller	명시적	표현식 함수로 간결
Null 처리	주의 필요	언어 차원의 nullable 타입
JPA 생태계	매우 성숙	Spring 지원 우수
학습자료	매우 많음	많음
기존 기업 시스템	강점	점차 증가
신규 Spring 프로젝트	매우 좋음	매우 좋음
코드량	상대적으로 많음	상대적으로 적음

12. 계층별 역할 비교

계층	역할	예제
Controller	HTTP 요청/응답	POST, GET, PUT, DELETE
DTO	API 데이터 전달	MemberDto
Service	업무 로직 및 트랜잭션 중심	create, update 등
Entity	DB 테이블 매핑	Member
Repository	DB 접근	JpaRepository
JPA/Hibernate	객체와 SQL 연결	INSERT/SELECT/UPDATE/DELETE 생성
Database	데이터 저장	MariaDB/PostgreSQL/Oracle

13. 실무형으로 발전시키는 순서

기본 CRUD 완성 → Request DTO / Response DTO 분리 → Bean Validation → 전역 예외처리 → HTTP 상태코드 정리 → 검색 → 페이징/정렬 → 트랜잭션 → 테스트 → Swagger/OpenAPI 순으로 확장하면 이해하기 쉬움.

실무에서는 비밀번호나 운영 DB 접속정보를 소스에 직접 저장하지 않고 환경변수나 비밀관리 방식을 사용하는 것이 바람직함.

14. 최종 핵심 암기

등록: POST → Controller → Service → DTO→Entity → repository.save() → INSERT

조회: GET → Controller → Service → repository.find...() → SELECT → Entity→DTO

수정: PUT → 기존 Entity 조회 → 값 변경 → save() → UPDATE

삭제:

```sql
DELETE → 존재 확인 → deleteById() → DELETE
```

부록 · Spring Boot — Java · Kotlin

Spring Boot Java + Kotlin Oracle + MariaDB Multi-Database 완성 교재

OracleConfig + MariaDbConfig · Entity · Repository · DTO · Service · Controller · CRUD

1. 전체 아키텍처

```text
REST Controller
      ↓
Service
      ↓
DTO
  ┌─────┴──────────┐
  ↓                ↓
Oracle Repo      MariaDB Repo
  ↓                ↓
Oracle Entity    MariaDB Entity
  ↓                ↓
Oracle XE        MariaDB
1521/XEPDB1      3306/wizdata
```

멀티 DB에서는 Oracle과 MariaDB에 각각 DataSource, EntityManagerFactory, TransactionManager를 구성하고 Repository 패키지를 분리하는 것이 핵심임.

2. 권장 프로젝트 구조

```text
src/main/java(or kotlin)/com/example/multidb/
├── MultiDbApplication
├── config/
│   ├── OracleConfig
│   └── MariaDbConfig
├── oracle/
│   ├── entity/OracleMember
│   └── repository/OracleMemberRepository
├── mariadb/
│   ├── entity/MariaMember
│   └── repository/MariaMemberRepository
├── dto/MemberDto
├── service/MemberService
└── controller/MemberController
```

3. Maven 의존성

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.oracle.database.jdbc</groupId>
    <artifactId>ojdbc11</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

Kotlin 프로젝트는 kotlin-reflect, kotlin-stdlib-jdk8 및 Spring/JPA 관련 Kotlin 플러그인을 추가함.

4. application.yml

```yaml
app:
  datasource:
    oracle:
      jdbc-url: jdbc:oracle:thin:@localhost:1521/XEPDB1
      username: devuser
      password: oracle_password
      driver-class-name: oracle.jdbc.OracleDriver

    mariadb:
      jdbc-url: jdbc:mariadb://localhost:3306/wizdata
      username: devuser
      password: mariadb_password
      driver-class-name: org.mariadb.jdbc.Driver

spring:
  jpa:
    open-in-view: false
    properties:
      hibernate:
        format_sql: true
```

5. Oracle + MariaDB 접속 비교 10개

#	항목	Oracle	MariaDB
1	JDBC URL	jdbc:oracle:thin:@localhost:1521/XEPDB1	jdbc:mariadb://localhost:3306/wizdata
2	기본 포트	1521	3306

#	항목	Oracle	MariaDB
3	Driver	oracle.jdbc.OracleDriver	org.mariadb.jdbc.Driver
4	서비스/DB	XEPDB1	wizdata
5	DataSource Bean	oracleDataSource	mariaDataSource
6	EMF	oracleEntityManagerFactory	mariaEntityManagerFactory
7	Transaction	oracleTransactionManager	mariaTransactionManager
8	Entity 패키지	oracle.entity	mariadb.entity
9	Repository	oracle.repository	mariadb.repository
10	JPA 구현	Hibernate/JPA	Hibernate/JPA

6. Java OracleConfig

```java
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = "com.example.multidb.oracle.repository",
    entityManagerFactoryRef = "oracleEntityManagerFactory",
    transactionManagerRef = "oracleTransactionManager"
)
public class OracleConfig {

    @Primary
    @Bean
    @ConfigurationProperties("app.datasource.oracle")
    public DataSource oracleDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Primary
    @Bean
    public LocalContainerEntityManagerFactoryBean oracleEntityManagerFactory(
            EntityManagerFactoryBuilder builder) {
        return builder
                .dataSource(oracleDataSource())
                .packages("com.example.multidb.oracle.entity")
                .persistenceUnit("oracle")
                .build();
    }

    @Primary
    @Bean
    public PlatformTransactionManager oracleTransactionManager(
            @Qualifier("oracleEntityManagerFactory")
            EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}
```

7. Java MariaDbConfig

```java
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = "com.example.multidb.mariadb.repository",
    entityManagerFactoryRef = "mariaEntityManagerFactory",
    transactionManagerRef = "mariaTransactionManager"
)
public class MariaDbConfig {

    @Bean
    @ConfigurationProperties("app.datasource.mariadb")
    public DataSource mariaDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean mariaEntityManagerFactory(
            EntityManagerFactoryBuilder builder,
            @Qualifier("mariaDataSource") DataSource dataSource) {
        return builder
                .dataSource(dataSource)
                .packages("com.example.multidb.mariadb.entity")
                .persistenceUnit("mariadb")
                .build();
    }

    @Bean
    public PlatformTransactionManager mariaTransactionManager(
            @Qualifier("mariaEntityManagerFactory")
            EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}
```

8. Kotlin OracleConfig

```kotlin
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = ["com.example.multidb.oracle.repository"],
    entityManagerFactoryRef = "oracleEntityManagerFactory",
    transactionManagerRef = "oracleTransactionManager"
)
class OracleConfig {

    @Primary
    @Bean
    @ConfigurationProperties("app.datasource.oracle")
    fun oracleDataSource(): DataSource =
        DataSourceBuilder.create().build()

    @Primary
    @Bean
    fun oracleEntityManagerFactory(
        builder: EntityManagerFactoryBuilder
    ): LocalContainerEntityManagerFactoryBean =
        builder
            .dataSource(oracleDataSource())
            .packages("com.example.multidb.oracle.entity")
            .persistenceUnit("oracle")
            .build()

    @Primary
    @Bean
    fun oracleTransactionManager(
        @Qualifier("oracleEntityManagerFactory") emf: EntityManagerFactory
    ): PlatformTransactionManager =
        JpaTransactionManager(emf)
}
```

9. Kotlin MariaDbConfig

```kotlin
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = ["com.example.multidb.mariadb.repository"],
    entityManagerFactoryRef = "mariaEntityManagerFactory",
    transactionManagerRef = "mariaTransactionManager"
)
class MariaDbConfig {

    @Bean
    @ConfigurationProperties("app.datasource.mariadb")
    fun mariaDataSource(): DataSource =
        DataSourceBuilder.create().build()

    @Bean
    fun mariaEntityManagerFactory(
        builder: EntityManagerFactoryBuilder,
        @Qualifier("mariaDataSource") dataSource: DataSource
    ): LocalContainerEntityManagerFactoryBean =
        builder
            .dataSource(dataSource)
            .packages("com.example.multidb.mariadb.entity")
            .persistenceUnit("mariadb")
            .build()

    @Bean
    fun mariaTransactionManager(
        @Qualifier("mariaEntityManagerFactory") emf: EntityManagerFactory
    ): PlatformTransactionManager =
        JpaTransactionManager(emf)
}
```

10. Oracle + MariaDB 테이블 생성 SQL 10개 비교

테이블	Oracle 핵심 SQL	MariaDB 핵심 SQL
MEMBER	ID NUMBER GENERATED BY DEFAULT AS IDENTITY, NAME VARCHAR2(100)	id BIGINT AUTO_INCREMENT, name VARCHAR(100)
PRODUCT	PRICE NUMBER(15,2)	price DECIMAL(15,2)
BOARD	CONTENT CLOB	content LONGTEXT
EMPLOYEE	SALARY NUMBER(12,2)	salary DECIMAL(12,2)
CUSTOMER	PHONE VARCHAR2(30)	phone VARCHAR(30)
ORDERS	CREATED_AT DATE DEFAULT SYSDATE	created_at DATETIME DEFAULT CURRENT_TIMESTAMP
PAYMENT	STATUS VARCHAR2(20)	status VARCHAR(20)
UPLOAD_FILE	FILE_PATH VARCHAR2(500)	file_path VARCHAR(500)
CATEGORY	NAME VARCHAR2(100)	name VARCHAR(100)
ACCESS_LOG	MESSAGE VARCHAR2(1000)	message VARCHAR(1000)

11. 대표 MEMBER 테이블 전체 SQL

Oracle

```sql
CREATE TABLE MEMBER (
  ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  NAME VARCHAR2(100) NOT NULL,
  EMAIL VARCHAR2(200),
  AGE NUMBER,
  CREATED_AT DATE DEFAULT SYSDATE
);
```

MariaDB

```sql
CREATE TABLE member (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(200),
  age INT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

12. Java Entity

```java
// OracleMember.java
@Entity
@Table(name="MEMBER")
public class OracleMember {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private Integer age;

    public OracleMember() {}
    // getters/setters
}

// MariaMember.java
@Entity
@Table(name="member")
public class MariaMember {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private Integer age;

    public MariaMember() {}
    // getters/setters
}
```

13. Kotlin Entity

```kotlin
@Entity
@Table(name="MEMBER")
class OracleMember(
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String? = null,
    var age: Int? = null
)

@Entity
@Table(name="member")
class MariaMember(
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String? = null,
    var age: Int? = null
)
```

14. Java Repository + DTO

```java
public interface OracleMemberRepository
        extends JpaRepository<OracleMember, Long> {
    List<OracleMember> findByNameContaining(String name);
}

public interface MariaMemberRepository
        extends JpaRepository<MariaMember, Long> {
    List<MariaMember> findByNameContaining(String name);
}

public record MemberDto(
    Long id,
    String name,
    String email,
    Integer age
) {}
```

15. Kotlin Repository + DTO

```kotlin
interface OracleMemberRepository :
    JpaRepository<OracleMember, Long> {
    fun findByNameContaining(name: String): List<OracleMember>
}

interface MariaMemberRepository :
    JpaRepository<MariaMember, Long> {
    fun findByNameContaining(name: String): List<MariaMember>
}

data class MemberDto(
    val id: Long?,
    val name: String,
    val email: String?,
    val age: Int?
)
```

16. JPA Entity → Repository → DTO 예제 10개 비교

#	기능	Java	Kotlin
1	전체 조회	repository.findAll()	repository.findAll()
2	ID 조회	repository.findById(1L)	repository.findById(1L)
3	저장	repository.save(member)	repository.save(member)
4	삭제	repository.deleteById(1L)	repository.deleteById(1L)
5	개수	repository.count()	repository.count()
6	이름검색	List<Member> findByName(String name)	fun findByName(name:String): List<Member>
7	이메일	Optional<Member> findByEmail(String email)	fun findByEmail(email:String): Member?
8	나이이상	findByAgeGreaterThanEqual(Integer age)	findByAgeGreaterThanEqual(age:Int)
9	포함검색	findByNameContaining(String name)	findByNameContaining(name:String)
10	정렬	findAll(Sort.by(DESC,"id"))	findAll(Sort.by(DESC,"id"))

17. Java Service - 두 DB 동시 사용

```java
@Service
public class MemberService {

    private final OracleMemberRepository oracleRepository;
    private final MariaMemberRepository mariaRepository;

    public MemberService(
            OracleMemberRepository oracleRepository,
            MariaMemberRepository mariaRepository) {
        this.oracleRepository = oracleRepository;
        this.mariaRepository = mariaRepository;
    }

    @Transactional("oracleTransactionManager")
    public OracleMember saveOracle(OracleMember member) {
        return oracleRepository.save(member);
    }

    @Transactional("mariaTransactionManager")
    public MariaMember saveMaria(MariaMember member) {
        return mariaRepository.save(member);
    }

    public List<MemberDto> oracleMembers() {
        return oracleRepository.findAll().stream()
            .map(m -> new MemberDto(
                m.getId(), m.getName(), m.getEmail(), m.getAge()))
            .toList();
    }

    public List<MemberDto> mariaMembers() {
        return mariaRepository.findAll().stream()
            .map(m -> new MemberDto(
                m.getId(), m.getName(), m.getEmail(), m.getAge()))
            .toList();
    }
}
```

18. Kotlin Service - 두 DB 동시 사용

```kotlin
@Service
class MemberService(
    private val oracleRepository: OracleMemberRepository,
    private val mariaRepository: MariaMemberRepository
) {
    @Transactional("oracleTransactionManager")
    fun saveOracle(member: OracleMember) =
        oracleRepository.save(member)

    @Transactional("mariaTransactionManager")
    fun saveMaria(member: MariaMember) =
        mariaRepository.save(member)

    fun oracleMembers(): List<MemberDto> =
        oracleRepository.findAll().map {
            MemberDto(it.id, it.name, it.email, it.age)
        }

    fun mariaMembers(): List<MemberDto> =
        mariaRepository.findAll().map {
            MemberDto(it.id, it.name, it.email, it.age)
        }
}
```

19. Java REST Controller

```java
@RestController
@RequestMapping("/api/members")
public class MemberController {

    private final MemberService service;

    public MemberController(MemberService service) {
        this.service = service;
    }

    @GetMapping("/oracle")
    public List<MemberDto> oracle() {
        return service.oracleMembers();
    }

    @GetMapping("/mariadb")
    public List<MemberDto> mariadb() {
        return service.mariaMembers();
    }
}
```

20. Kotlin REST Controller

```kotlin
@RestController
@RequestMapping("/api/members")
class MemberController(
    private val service: MemberService
) {
    @GetMapping("/oracle")
    fun oracle() = service.oracleMembers()

    @GetMapping("/mariadb")
    fun mariadb() = service.mariaMembers()
}
```

21. Java ↔ Kotlin 핵심 비교

항목	Java	Kotlin
DTO	record/class	data class
Repository	extends JpaRepository	: JpaRepository
생성자 주입	생성자 작성	주생성자로 간단히 처리
Null	Optional/@Nullable 등	타입에 ? 표시
Entity	getter/setter 필요	프로퍼티 중심
Service	코드량 상대적으로 많음	간결함
Transaction	@Transactional("...")	@Transactional("...")
JPA 원리	동일	동일
Oracle/MariaDB 분리	Config/Package 분리	Config/Package 분리
추천	기존 Java 프로젝트	신규 간결한 Spring 프로젝트

22. 실행 및 테스트 순서

1) Oracle Docker 1521 및 MariaDB Docker 3306이 실행 중인지 확인함.

2) Oracle XEPDB1에 devuser 계정과 MEMBER 테이블을 준비함.

3) MariaDB wizdata에 devuser 계정과 member 테이블을 준비함.

4) application.yml의 IP, 사용자, 비밀번호를 실제 서버 환경에 맞춤.

5) Spring Boot를 실행함.

6) GET /api/members/oracle 로 Oracle 데이터를 확인함.

7) GET /api/members/mariadb 로 MariaDB 데이터를 확인함.

8) 저장 메서드에서는 사용할 DB에 맞는 TransactionManager를 명시함.

23. 실무 핵심 주의사항

- 멀티 DataSource에서는 @Primary를 한쪽(본 교재에서는 Oracle)에 명확히 지정함.

- Oracle Repository와 MariaDB Repository의 basePackages가 겹치지 않게 분리함.
- 각 EntityManagerFactory의 packages()가 해당 DB Entity 패키지만 스캔하도록 설정함.
- 한 메서드에서 Oracle과 MariaDB를 동시에 수정하는 경우 단일 로컬 @Transactional만으로 두 DB 전체의 원자성을 자동 보장하지 않음. 분산 트랜잭션이 필요한지는 별도로 설계해야 함.
- 운영 비밀번호를 소스코드나 Git 저장소에 직접 넣지 말고 환경변수/Secret 방식 사용을 권장함.
- Oracle과 MariaDB의 IDENTITY, 날짜형, CLOB/LONGTEXT 등 SQL 차이를 확인함.
